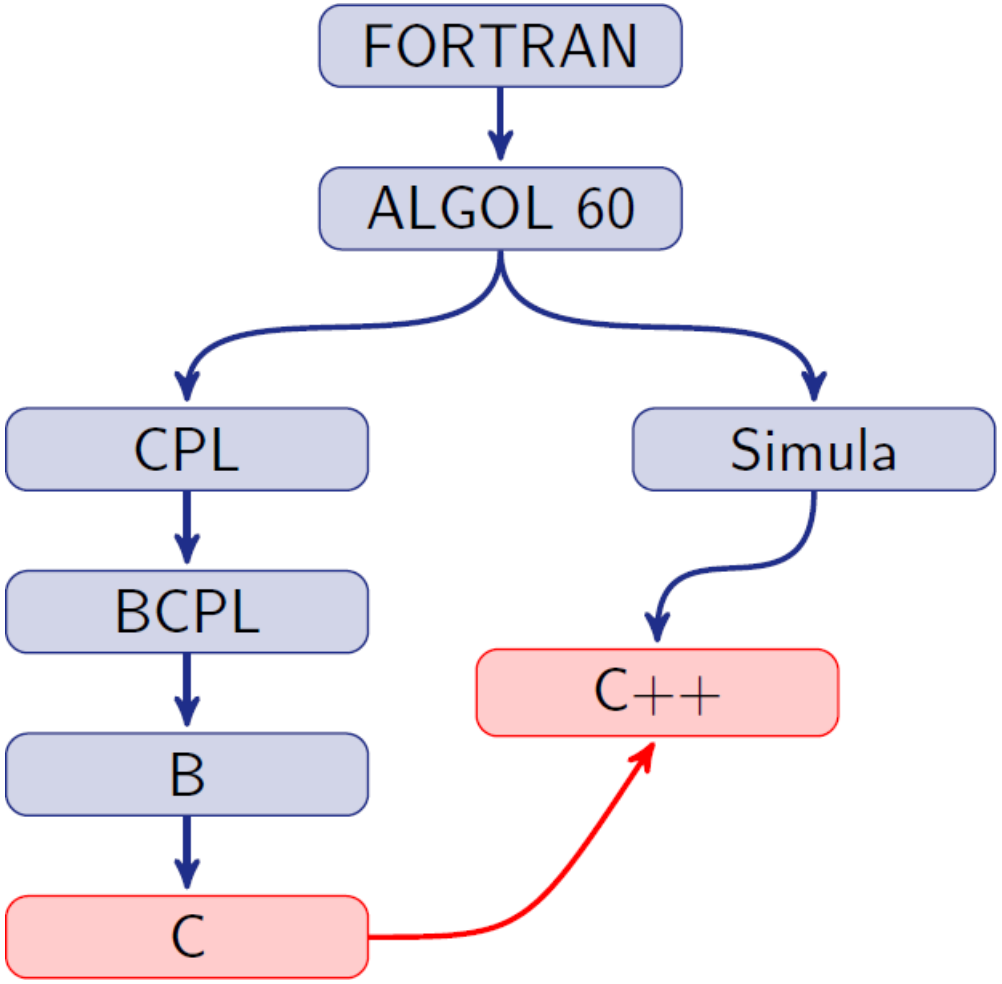
University of Exeter

ECM2433: The C Family Moving to C++

Overview

- C++ background
  - Some history
  - Compiling and linking
  - Standards
- Getting started in C++
  - Hello World!
  - Standard streams
  - Boolean
  - String
  - Keywords using and auto
  - Function overloading
  - Class definitions

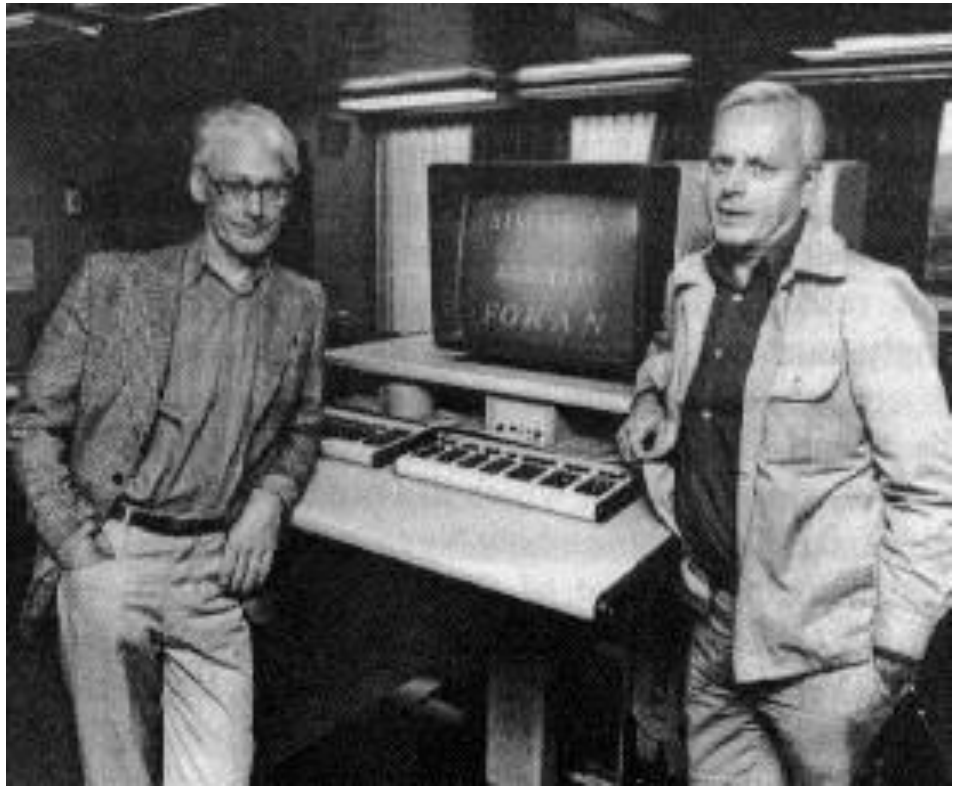
C++: some history



Simula: 1964

Developed by Ole-Johan Dahl and Kristen Nygaard
at the Norwegian Computing Center, Oslo

- Extension of ALGOL 60
- First object-oriented language
  Simula 67 introduced:
  - objects
  - classes
  - inheritance and subclasses



www.mn.uio.no/ifi/om/minnesider/

Kristen Nygaard and Ole-Johan Dahl: “The Development of the SIMULA Languages", ACM, 1978.

“C with Classes”

C makes it easy to shoot yourself in the foot; C++ makes it harder, but when you do it blows your whole leg off.

Bjarne Stroustrup, ca 1986

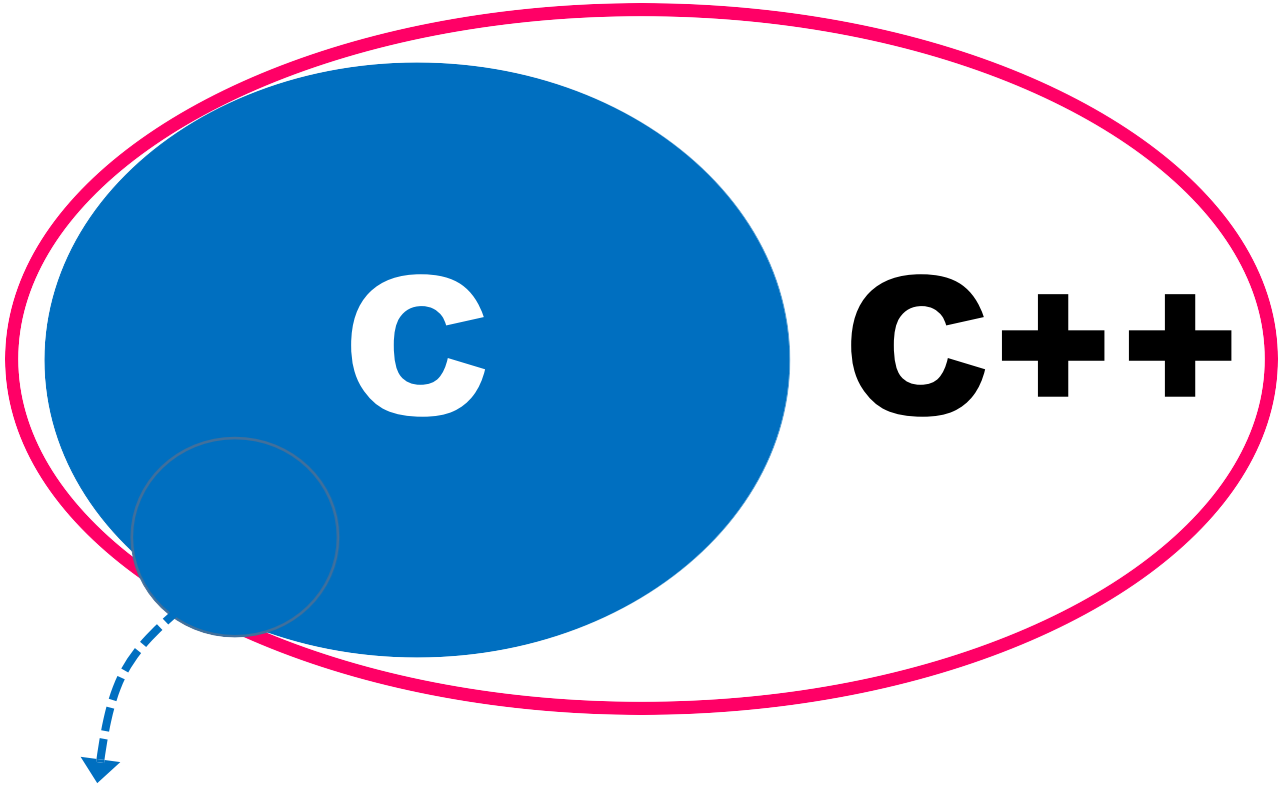
*In October of 1979, I had a **preprocessor**, called Cpre, that added Simula-like classes to C; and in March of 1980, this preprocessor had been refined to the point where it supported one real project and several experiments. [...] The first key C++ library, the task system supporting a **co-routine** style of programming, was crucial to the usefulness of "C with Classes," as the language accepted by the preprocessor was called, in these projects.*

“A History of C++: 1979−1991,” Bjarne Stroustrup. Proceedings ACM History of Programming Languages Conference (HOPL-2). ACM Sigplan Notices. Vol 28 No 3, pp. 271-298. March 1993.

Bjarne Stroustrup at Bell Labs

https://www.stroustrup.com/

C++ is (very nearly) a superset of C



C++ adds:

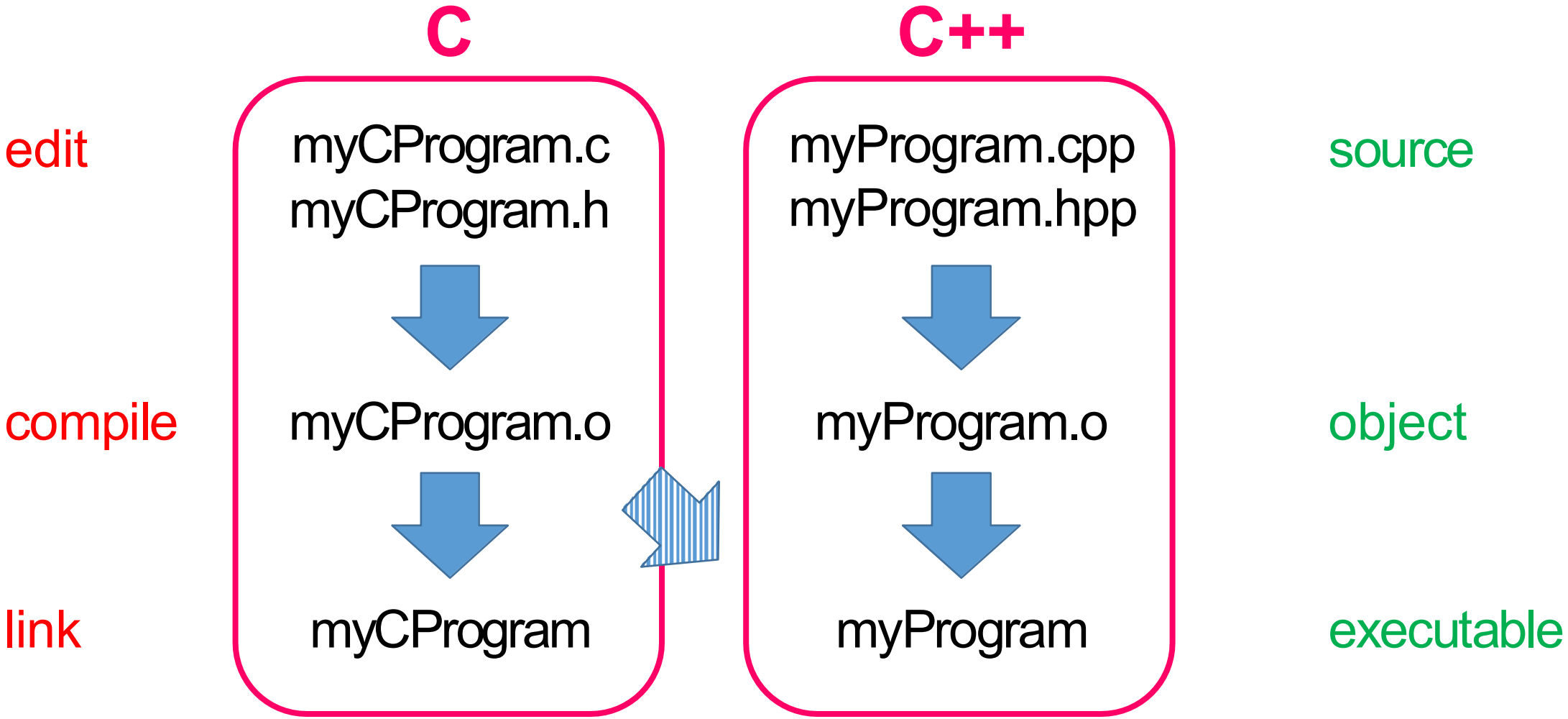
- classes
- exception handling
- function & operator overloading
- default values for parameters
- pass by reference

Implicit casting of `void *`

C99: variable length arrays; native complex number types

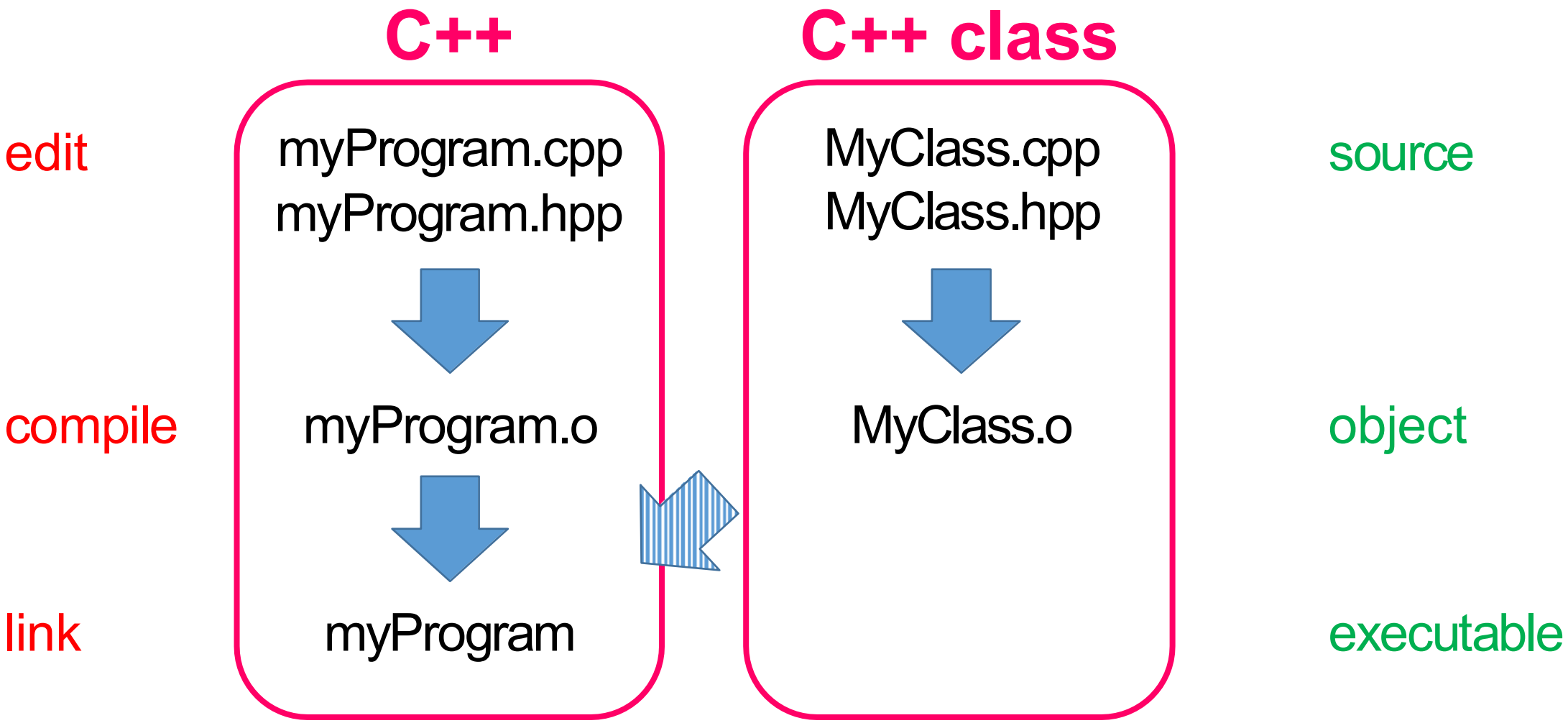
For differences see: https://en.wikipedia.org/wiki/Compatibility_of_C_and_C%2B%2B

Compiling and linking



Header files may be *.h for C++.
Some (e.g. Boost library) prefer *.hpp

Compiling and linking



GNU C++ compiler

ANSI standard **C** compile:

```
gcc -ansi -c myCProgram.c -o myCProgram.o
```

C++ compile:

```
g++ -std=c++17 -c myCPPProgram.cpp -o myCPPProgram.o
```

and link:

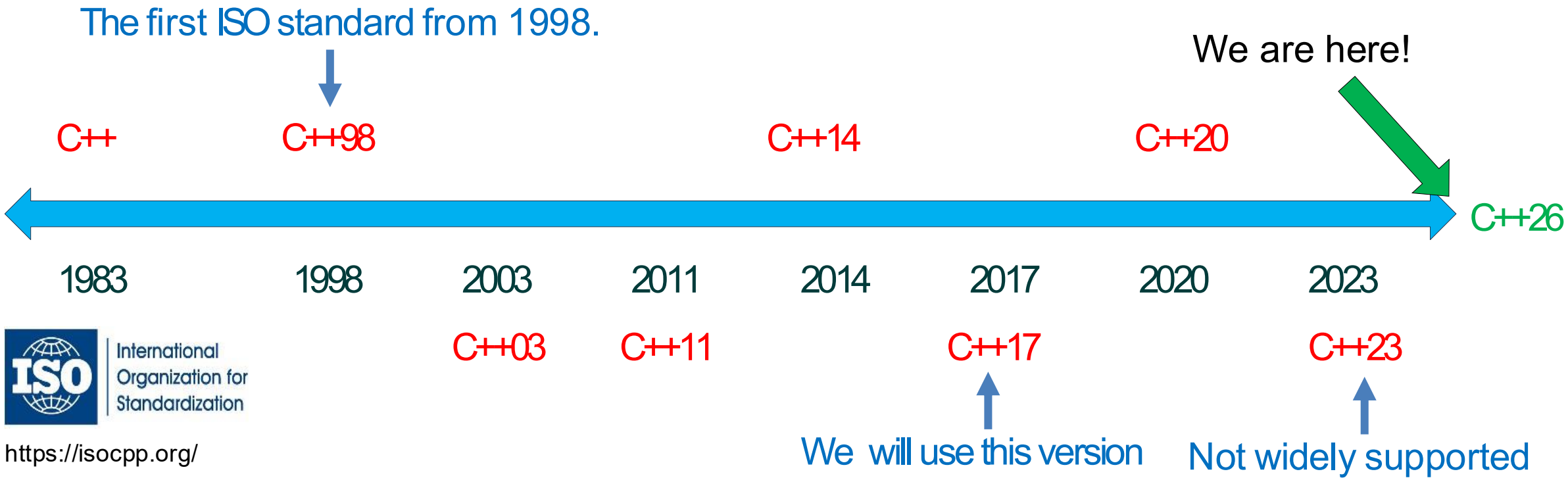
```
g++ myCProgram.o myCPPProgram.o -o myCPPProgram
```

C++ standards

Standard compile:

```
g++ -std=c++17 -c myCPPProgram.cpp -o myCPPProgram.o
```

The ISO standards for C++ have developed over time:



International Organization for Standardization

https://isocpp.org/

Hello World! again

```
/* My example hello world program */
// The C++ version

#include <stdio.h>
#include <iostream>

int main()
{
   printf("Hello world!\n");
   std::cout << "Hello world!" << std::endl;
   return 0;
}
```

scope resolution operator

Hello World! again

```
/* My example hello world program */
// The C++ version

#include <stdio.h>
#include <iostream>
using namespace std;

int main()
{
   printf("Hello world!\n");
   cout << "Hello world!" << endl;
   return 0;
}
```

removes the need for the scope resolution operator here

cout and cerr

```
#include <iostream>
using namespace std;

int main()
{
    int   elephantCount = 42;

    cout << "Count: " << elephantCount << "\n";
    cerr << "Count: " << elephantCount << endl;

    return 0;
}
```

goes to stdout

goes to stderr

output operator ("put to")

only prints a newline

prints a newline and flushes the output buffer

cout formatting

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
   int   aNum = 42;

    cout << setfill('*') << setw(10) << aNum << ";" << aNum << '\n';
    cout << setbase(16) << aNum << ";" << setbase(8) << aNum << '\n';
    cout << hex << aNum << ";" << oct << aNum << endl;

   return 0;
}
```

<iomanip> provides manipulators that take arguments (e.g. `setw`, `setfill`)

```
********42;42
2a;52
2a;52
```

cin

```
#include <iostream>
using namespace std;
int main()
{
    int    i;
```

goes to stdin

```
    cin >> i;
```

read an integer into i

```
    double d;
    cin >> d;
```

read a double-precision floating point number into d

intput operator (“get from”)

```
    return 0;
}
```

C++ Boolean type

```
#include <iostream>

int main()
{
    bool flag = false;
    bool truthy = true;
    std::cout << "flag = " << flag << "\n";
    std::cout << "truthy = " << truthy << "\n";

    std::cout << std::boolalpha; // Enable textual representation of bools
```

No <stdbool.h> needed in C++

```
flag = 0
truthy = 1
```

Casting: Non-zero values become true

```
    flag = -5.4;
    std::cout << "flag = " << flag << "\n";
```

Casting: zero value becomes false

```
    flag = 0;
    std::cout << "flag = " << flag << "\n";

    return 0;
}
```

```
flag = true
flag = false
```

C++ std::string

University of Exeter

- std::string is a **class** in C++
- It wraps a dynamic char array and manages memory **automatically**

C way

```
#include <string.h>


int main() {

char s1[10] = "Hello";
char s2[] = " world";


// s1 = “hello”; // Error! Cannot assign
strcpy(s1, “hello”);
strcmp(s1, s2)
strcat(s1, s2);
return 0;
}
```

Crash, s1 size (10) < "hello world" +\0 (12)

C++ way

```
#include <string>


int main() {

std::string s1 = "Hello"; // Dynamic size
std::string s2 = " world";


s1 = "hello";
if (s1 == s2) {}
s1 = s1 + s2;


return 0;
}
```

assignment, deep copy automatically

natural comparison

safe concatenation (auto-resize)

std - The C++ Standard Library

- Built-in types, functions, and more provided by C++
- You need to #include the relevant file
  - #include <string> → std::string
  - #include <iostream> → std::cout, std::endl
- We prefix standard library names with std::
  - If we write using namespace std; this is considered a bad style as it brings many names into the global scope and may cause name conflicts.

using keyword

Similar to **typedef** in C, but provides clearer syntax and supports **template** aliases in C++. (We'll learn more about template later)

```
using your_defined_name = the actual datatype
```

```
using ulong = unsigned long
```

=

```
typedef unsigned long ulong;
```

```
template <typename T>
using MyVector = std::vector<T>;
```

```
template <typename T>
typedef std::vector<T> MyVector;
```

not allowed

```
MyVector<int> v1; // same as std::vector<int>
```

```
std::vector<std::pair<int, std::string>> grades;
```

without using

```
using Grade = std::pair<int, std::string>;
using GradeList = std::vector<Grade>;

GradeList grades;
```

with using

auto keyword

University of Exeter

`auto` means: let the compiler figure out the type.

```
auto x = 10;          // int
auto y = 3.14;        // double
auto z = x + y;       // double
auto ch = 'c';        // char
auto val = sqrt(y)    // val has the type of whatever sqrt(y) returns
```

```
auto i = 1;        // int
i = "hello!";      // Doesn't compile
```

auto is still statically typed!

auto is most useful when the type is long, obvious from context, or not particularly important to the reader

Function overloading

```
int myabs (int i)
{
    if (i < 0)
        return -i;
    else
        return i;
}

float myabs (float f)
{
    if (f < 0.0)
        return -f;
    else
        return f;
}
```

must have different numbers or types of parameters

may have different return types

```
#include <iostream>
using namespace std;

int main()
{

    int   i = -3;
    float f = -4.6;

    cout << myabs(i) << endl;
    cout << myabs(f) << endl;
    return 0;
}
```

Classes: declaration

University of Exeter

Stack.h

```
#define SIZE 100


// Define the Stack class
class Stack
{
        int     stck[SIZE];
        int     index;
    public:
        void    push(int);
        int     pop(void);
};
```

private class variables

public member function prototypes

Classes: access specifiers

```
class class-name
{
    private variables and functions
  access-specifier:
    variables and functions
  access-specifier:
    variables and functions
  access-specifier:
    variables and functions
};
```

- private (default)
  can only be accessed within the class
- public
  can be accessed by any part of the program
- protected
  accessed by inheritors; details later

Classes: member function definitions

Stack.cpp

```
#include <Stack.h>

void Stack::push(int i)
{ ... };

int Stack::pop()
{ ... };
```

scope resolution

Stack.h

```
#define SIZE 100


// Define the Stack class
class Stack
{
        int     stck[SIZE];
        int     index;
    public:
        void    push(int);
        int     pop(void);
};
```

Using classes

```
#include <Stack.h>
int main ()
{
    Stack    s;
    int      value;

    s.push(23);
    s.push(1);
    value = s.pop();

    return 0;
}
```

Stack.h

```
#define SIZE 100

// Define the Stack class
class Stack
{
        int     stck[SIZE];
        int     index;
    public:
        void    push(int);
        int     pop(void);
};
```

Summary

- C++ is a **near-superset of C** that integrates Simula-inspired object-oriented features.
- Standard streams (cout, cin, cerr) provide a type-safe alternative to traditional C I/O.
- std::string manages memory automatically, offering safer string operations than C
- Modern keywords using and auto improve code readability and allow for automatic type deduction.
- Function overloading enables multiple functions to share a name if their parameters differ
- Classes encapsulate data and behavior, using public, private and protected access specifiers to protect data.

- Read about **std::string member functions** and **std::string_view**